

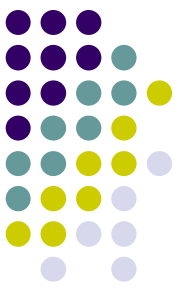
Singleton Pattern

Architecture of Computer Games
SE 456

Ed Keenan

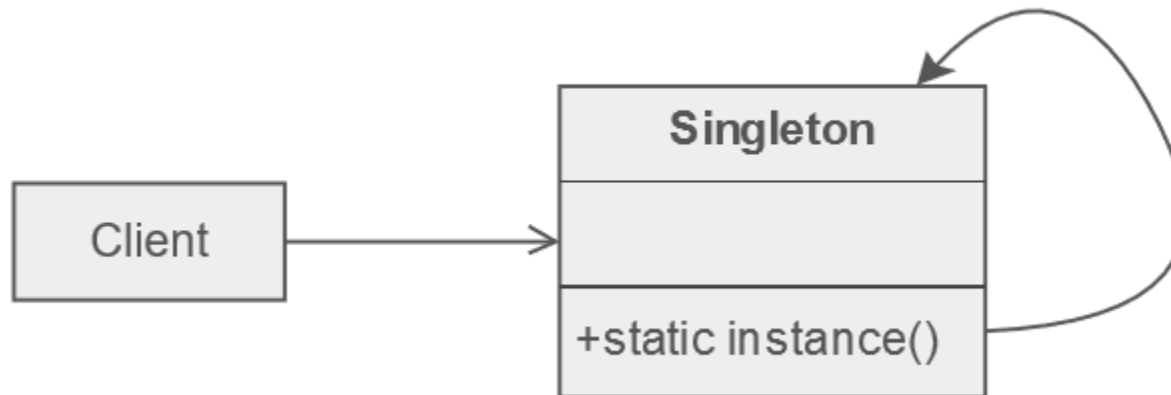
12 January 2022

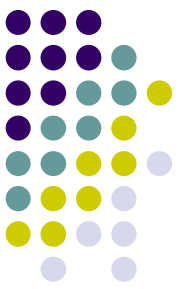
13.0.9.3.9 Mayan Long Count



Singleton

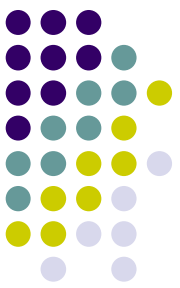
- Intent
 - Ensure a class has only one instance, and provide a global point of access to it.
- Remark
 - Encapsulated "**just-in-time initialization**" or "**initialization on first use**".





Singleton

- Singleton Pattern ensures a class has
 - only one instance
 - provides a global point of access to it



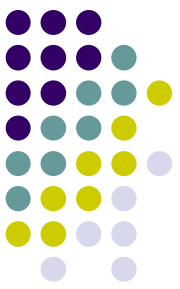
Singleton

The `getInstance()` method is static, which means it's a class method, so you can conveniently access this method from anywhere in your code using `Singleton.getInstance()`. That's just as easy as accessing a global variable, but we get benefits like lazy instantiation from the Singleton.

Singleton
static <code>uniqueInstance</code>
// Other useful Singleton data...
static <code>getInstance()</code>
// Other useful Singleton methods...

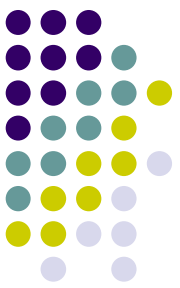
The `uniqueInstance` class variable holds our one and only instance of Singleton.

A class implementing the Singleton Pattern is more than a Singleton; it is a general purpose class with its own set of data and methods.



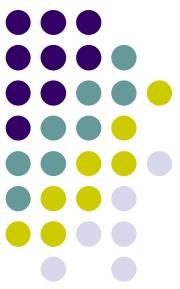
Singleton

- Lazy Instantiation
 - Key to this pattern
 - It isn't initialize until the **first time** you use it.
- Somewhere this is an Instantiation
 - Static or dynamic
 - **no-one can ever delete it**
- DRY Principle – **D**on't **R**epeat **Y**ourself
 - Only one copy
 - Unique



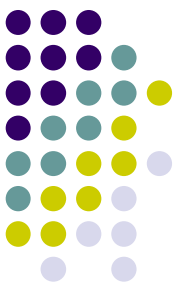
Singleton

- Mathematical concept of a singleton
 - **Restricting** the **instantiation** of a class to one object.
 - Useful when exactly one object is needed to coordinate actions across the system.
 - Generalized
 - Systems that operate more efficiently when only one object exists
 - Restrict the instantiation to a certain number of objects
 - Anti-pattern
 - Judging that it is **overused**, introduces unnecessary limitations in situations where a sole instance of a class is not actually required, and introduces global state into an application



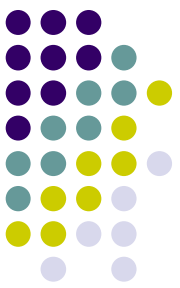
Singleton

- Singletons are often **preferred to global variables**
 - Don't pollute the global name space with unnecessary variables.
- They permit lazy allocation and initialization
 - global variables will always consume resources



Singleton

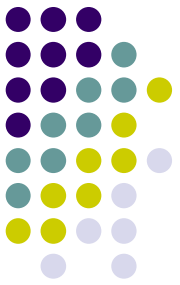
- When to use it?
 - Single system manager in a game
 - Input manager
 - Sprite manager
 - Score, number of Lives, etc...
 - Anything that follows the pattern
 - Create once a game
 - Use many times
 - Destroy once a game



Singleton

- HFDP - Chapter 5 – Singleton Pattern
 - Head First Design Patterns book
 - Campy – but get's the message across
- Reference Design Patterns
 - DoFactory – Design Patterns in C#
 - Try those demos
- Code Demo

Thank You!



- Questions?

